Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

Agentic AI for Code Generation

Seminar Report

Student: Thua Duc Nguyen ✉ **and Supervisor: Derui Zhu** ✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ thuaduc.nguyen@tum.de
✉ derui.zhu@tum.de

June 30, 2026

Abstract — Large language models have moved from autocompleting single functions to resolving real software issues on their own. This report surveys the agents that do so through a single lens: a five-stage *agent pipeline* of intent capture, planning, code generation, execution, and refinement. We read three lines of work against that pipeline. First, end-to-end agents (SWE-agent, Agentless, AutoCodeRover, OpenHands, Devin) and the patterns they share. Second, planning and search, where we argue that planning contributes *grounding* rather than deliberation. Third, execution feedback, whose mechanisms we sort by what closes their loop: an authoritative oracle, a learned or generated proxy, or the generator's own critique. We close on evaluation realism and cost.

1 Introduction

Motivation. Coding with a language model used to mean accepting an autocomplete suggestion. Increasingly it means describing a change in plain English and letting a system find the right files, edit them, run the tests, and iterate until they pass. The appeal and the worry are the same fact: the developer states an intent and reviews an outcome, while the steps in between (reading the codebase, forming a hypothesis, running the debugger) are delegated to the model. The measured progress is real. In late 2023 there were no agents to speak of: the strongest result on SWE-bench, a benchmark of real GitHub issues, came from retrieval-augmented prompting of a frontier model and resolved under 2% of them [22]. Two years later agents resolve most of the human-verified split. This report asks what produced that jump, and what it still does not deliver.

Background and Related Surveys. Existing surveys catalogue agents in software engineering [43] and LLM-based code-generation agents [12], organised largely by system or by capability. This report instead organises three active research fronts (end-to-end agents, planning and search, and execution-feedback refinement) around the pipeline stages they instantiate, which lets us compare systems that describe themselves in incompatible vocabularies.

Scope and Contribution. We cover agents whose output is a code change: repository-level issue resolution, automated program repair, and function-level synthesis, drawing on the 2023–2026 literature. Editor autocompletion and code search are out of scope. So are multi-agent *role-play* frameworks that simulate a development team (ChatDev [35], MetaGPT [18]): what they add is an organisational metaphor, not a way of grounding a patch. Multi-agent *search and decomposition* pipelines such as SWE-Search [3] do stay in scope, since they produce code changes; we treat them as variants of the search strategies of §4 rather than as a separate architectural family. The report makes three contributions. It offers the agent pipeline as a shared vocabulary for these systems (§2). It surveys the three fronts against that pipeline (§3–§5). And it consolidates what the fronts share and where they are jointly stuck (§6), drawing on 2025–2026 evidence about benchmark contamination and cost that postdates most existing surveys.

2 The Agent Pipeline

2.1 Five Stages

Papers on autonomous coding agents describe their systems in their own terms: phases, modules, roles, loops. Underneath, nearly all do the same five things. Naming those stages gives us a frame for comparing systems that otherwise share no vocabulary. The pipeline is a lens on existing architectures, not a new one. Its value is that it lets us ask one question of every system at once: at each stage, what can tell the model it is wrong? The five stages loosely resemble the classical waterfall phases, but the loop breaks the

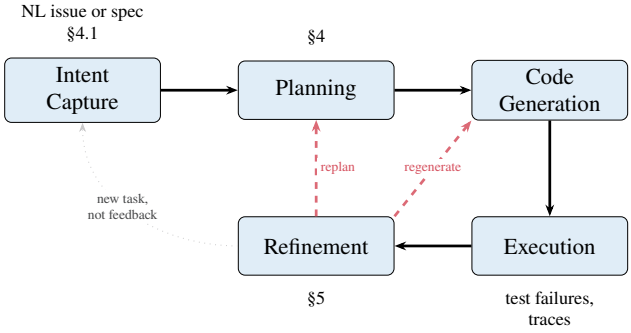

Figure 1 The agent pipeline. Intent flows through planning, generation, and execution. Execution failures drive refinement (dashed), which either regenerates the patch or replans when the initial hypothesis is invalidated. Note that *no feedback edge re-enters intent capture*: the dotted return edge starts a fresh task rather than correcting the intent of the current one. That asymmetry is deliberate and is the subject of §4.1.

resemblance. An agent runs through all five in seconds and repeats them many times per task, and the cycle is closed by a test runner rather than a review meeting. The closer analogue is the developer's inner loop of edit, run, and debug, with planning and localisation folded in.

The agent pipeline (Figure 1) consists of five stages forming a loop: Intent Capture → Planning → Code Generation → Execution → Refinement. Execution feedback drives refinement, which may trigger re-planning or re-generation. Adjacent stages exchange structured artefacts: natural-language intent and repository context flow into plans and candidate patches; generated code is run against test oracles; failures and traces feed back into replanning or repair.

2.2 Mapping to Survey Topics

What makes the five-stage division load-bearing for the argument, rather than for the table of contents, is what each stage *does*. Read as a control system: intent capture is the uncontrolled input, generation is the plant, execution is the oracle, and planning and refinement are the control layer that ties the plant to the oracle. The pipeline is therefore a map of where oracles are and are not. The two stages with none behind them, intent capture and validation whose oracle is a weak test suite, turn out to be exactly the two places the field is stuck (§6).

Each of the three main sections covers one or more stages. §3 treats the full pipeline as an integrated system, including the action interfaces used for code generation (ACI, code-as-action, diff-based edits). §4 treats intent capture (§4.1) together with planning, candidate search, and the plan–generate interface, since in practice an agent reads the intent and plans over it in the same breath. §5 covers execution environments, test-driven feedback, and the refinement loop, including fault-specific self-repair. Code generation gets no section of its own: no surveyed system treats emitting tokens as a separable problem, and the decisions that govern it (action space, edit format, candidate sampling) belong to the stages on either side. It is therefore spread across §3 and §4. Cross-cutting concerns (evaluation, context management) are collected in the Discussion.

3 End-to-End Autonomous Software Engineering Agents

Before examining individual stages, we survey systems that orchestrate the *entire* pipeline, accepting a natural-language issue and producing a tested code change with minimal human intervention. These end-to-end agents are the setting in which planning (§4) and feedback-driven refinement (§5) operate. We identify the architectural patterns common to them (§3.1), survey five systems spanning the design space from fully autonomous to deliberately non-agentic (§3.2), and examine how they are evaluated (§3.3).

3.1 Architecture and Orchestration

Despite surface-level diversity, autonomous SE agents converge on three patterns, governing how they perceive the environment, what actions they may take, and how they manage repository-scale context.

The observe–think–act loop. The dominant paradigm is a ReAct-style [50] loop alternating between (i) observing the environment (file contents, test output, error traces), (ii) reasoning about the next step, and (iii) executing an action via a tool call. SWE-agent [48] and OpenHands [41] both instantiate it, differing mainly in the *action space* exposed to the model.

Table 1 Comparison of end-to-end autonomous SE agents. Architectural patterns: **OTA** = observe–think–act loop, **ACI** = custom Agent–Computer Interface, **CaA** = code-as-action, **Pipe** = fixed multi-phase pipeline (no agent loop). Splits are defined in Table 2. Scores are single-pass (pass@1) on each system's base configuration, so the ablations and pass@k variants in the text are higher; cost is average USD per instance on Lite. All figures are as-published by each system's authors and reflect that publication's backbone (Agentless's row is the revised 1.5 release). **The rows do not share a backbone and are not a controlled comparison**; see §3.2.

System	Architecture	Backbone	Context Strategy	Lite	Full	Cost
SWE-agent [48]	OTA + ACI	GPT-4 Turbo	Observation compression	18.0%	12.5%	$1.67
Agentless [46]	Pipe	GPT-4o	Repo skeleton + embeddings	32.0%	–[†]	$0.70
AutoCodeRover [51]	OTA + AST search	GPT-4	AST APIs (SBFL optional)	19.0%	12.4%	$0.43
OpenHands [41]	OTA + CaA	Claude 3.5 Sonnet	Docker sandbox + delegation	26.0%	–[†]	$1.10
Devin [9]	OTA + Brain/DevBox	undisclosed	Persistent knowledge store	–	13.9%[‡]	–[‡]

[†]Not evaluated on Full; the authors cite compute cost. [‡]Devin is closed-source: the backbone model and per-instance cost are undisclosed, and the resolve rate is self-reported on a random 25% subset of Full. No peer-reviewed paper exists and the result has not been independently reproduced.

The action interface. Yang et al. [48] argue that LLMs are a "new category of end users" and need a purpose-built *agent–computer interface* (ACI) rather than raw shell access. SWE-agent's ACI has four parts: a windowed file viewer (100 lines), a linter-guarded `edit` command, summarised search results, and a compressed observation history. Their ablations show each part earns its place. On Lite, removing the linter costs 3 percentage points, switching to iterative search costs 6 pp, and showing full files instead of a 100-line window costs 5.3 pp. The opposite design is *code-as-action*: OpenHands adopts the CodeAct paradigm [40] and folds every action into executable Python and bash inside a Docker sandbox. That buys flexibility, but leaves the model a larger action space to navigate.

Context management. Repository-level tasks easily exceed LLM context windows, and each system mitigates differently. SWE-agent collapses observations older than five turns. AutoCodeRover [51] parses the codebase into an AST and exposes search APIs that expand context coarse-to-fine, `search_class` returning a class skeleton of signatures and `search_method_in_class` the full body of one method once the agent has narrowed to it. Agentless [46] builds a hierarchical repository skeleton and retrieves candidate files by embedding before prompting.

3.2 Representative Systems

We survey five prominent systems that span the design space. Table 1 provides a comparative overview.

SWE-agent [48]. SWE-agent wraps a GPT-4 Turbo backbone in a custom ACI with specialised file navigation, editing, and search commands, achieving 12.47% on the full SWE-bench test set and 18.00% on Lite when first published. Its trajectory is typical: reproduce the issue, localise via `find_file`/`search_dir`, then enter an edit–execute loop. Runs average $1.67 per instance on Lite, but the distribution is bimodal: successful trajectories terminate quickly (a median of 12 steps, $1.21), whereas failures run to the step limit before giving up (a mean of 21 steps, $2.52).[1]

Agentless [46]. Agentless challenges the need for agentic autonomy. It replaces the interactive loop with a fixed three-phase pipeline: *localise* → *repair* → *validate*. Localisation narrows hierarchically (files → classes/functions → edit locations), combining LLM prompting with embedding retrieval. Repair samples 40 candidate patches per bug in a *search/replace* diff format, which anchors each edit to the surrounding source text rather than to a line number; the authors adopt it because it is cheaper and less prone to hallucination than emitting whole files. Validation then picks among the candidates using regression tests, LLM-generated reproduction tests, and majority voting over AST-normalised candidates. On GPT-4o it resolved 32.00% of Lite at $0.70 per problem, the best open-source figure at the time and a strong non-agentic baseline.[2] OpenAI subsequently adopted

[1]The two statistics are as reported by the authors and are not directly comparable: the success figure is a median and the failure figure a mean. The qualitative point (failures cost more than successes, so a low average conceals an expensive tail) does not depend on the choice.

[2]These are the figures of the revised release (Agentless 1.5, October 2024), carried in Table 1; the originally published version

Agentless as the scaffold for reporting GPT-4o and o1 results on Verified, describing it as the best-performing open-source scaffold available [31].

AutoCodeRover [51]. AutoCodeRover operates on program representations rather than raw files, parsing the codebase into an AST and exposing structure-aware search APIs; a stratified retrieval process expands context from issue keywords through successive search rounds. Where a test suite exists, Spectrum-Based Fault Localisation (SBFL) augments the search with suspiciousness scores. On a GPT-4 backbone it resolved 19.0% of Lite and 12.4% of Full in a single pass. Two extensions raise this without changing the backbone: SBFL lifts Lite to 22%, uniquely solving 7 instances no other configuration reaches, while pass@3 over three sampled runs gives 26.0% on Lite and 18.0% on Full, at $1.30 per task against $0.43 at pass@1.

OpenHands [41]. Formerly OpenDevin, OpenHands is an open-source platform (ICLR 2025) built around the CodeAct agent. Each session runs in a Docker container with a bash shell, IPython server, and browser, and a generalist agent may delegate to specialised ones. CodeAct v1.8 with Claude 3.5 Sonnet achieves 26.0% on Lite; a later submission reaches 60.6% on Verified with a single rollout and 66.4% when five trajectories are reranked by a dedicated critic model [33].

Devin (Cognition) [9]. Devin is a proprietary cloud-hosted agent separating a *Brain* (reasoning) from a *DevBox* (execution VM with shell, IDE, and browser), with a persistent knowledge store for cross-session memory. It is the one system surveyed here with no accompanying paper: the only primary source is a launch blog post reporting 13.86% resolved on SWE-bench. That is an order of magnitude above the 1.96% retrieval-augmented baseline and is the result that launched the agentic turn, but it was measured on a random 25% subset, with no released trajectories or harness, and has never been independently reproduced. The one longitudinal external assessment, a month-long trial by Answer.AI, found Devin completed 3 of 20 tasks with 14 outright failures [2]. We include Devin for its architectural influence, not as a comparable data point; its row in Table 1 is a vendor claim, not a measurement.

reported 27.33% at $0.34 on the same backbone. The comparison below is unaffected, since the matched-backbone baselines come from the same revision.

Taken together, these five expose a trade-off between agentic flexibility and pipeline efficiency. Agentless shows that a fixed localise–repair–validate pipeline can beat interactive agents on Lite at lower cost, which challenges the assumption that autonomy is needed for strong results. The claim needs care. The rows of Table 1 do not share a backbone (Agentless runs on GPT-4o, SWE-agent's published figure on GPT-4 Turbo), and §3.3 shows that swapping the backbone alone can move a resolve rate by ten points. The claim survives only because Agentless's authors also report the matched comparison, running the agentic baselines on the same GPT-4o backbone, where the gap persists [46]. It is that matched comparison, not the cross-backbone table, that carries the argument. Among agentic designs, interface and context strategy matter as much as the backbone: SWE-agent's ACI ablations, OpenHands's CodeAct action space, and AutoCodeRover's AST search APIs each move resolve rates by several percentage points at comparable model capacity.

3.3 Full-Pipeline Evaluation

SWE-bench. The primary evaluation instrument is SWE-bench [22]: 2,294 merged pull requests scraped from 12 Python repositories, retaining only PRs that both resolve a linked GitHub issue and modify the test suite. The two filters do different work. The test-suite filter supplies the grading oracle; the issue-linkage filter supplies the task's *intent*, and is the reason an agent has any natural-language input at all. An agent receives the repository snapshot from *before* the merge together with the issue text; its patch is scored against *fail-to-pass* tests (which must flip) and *pass-to-pass* tests (which must not regress). The headline metric throughout is the *resolve rate*: the percentage of instances where both sets pass on the first submitted patch (pass@1).

Terminology and comparability. "SWE-bench" denotes a family of task sets rather than a single benchmark (Table 2). Two distinctions warrant emphasis. Verified is the most carefully *curated* split, not the most demanding one: human screening removed underspecified issues and unfair tests, so its resolve rates run systematically higher than Full's or Lite's, and scores are not comparable across splits. Second, *easy*, *medium*, and *hard* are not splits but annotator difficulty tiers within Verified, defined by the time a human en-

Table 2 The SWE-bench family. Lite and Verified are subsets of Full and share its 12 Python repositories; Multilingual and Pro are independently constructed task sets that reuse the harness. Resolve rates are *not* comparable across rows.

Split	Size	Selection criterion
Full	2,294	All scraped PRs that touch tests
Lite	300	Single-file patches, self-contained issues; some issue texts leak the solution [46]
Verified	500	Human-screened for issue clarity and test fairness [31]
Multilingual	300	Same pipeline, 9 non-Python languages [37]
Pro	1,865	Long-horizon multi-file changes; copyleft and held-out proprietary repos [11]

gineer was estimated to require (<15 min, 15 min–1 h, >1 h) [15].

Leaderboard trajectory. At launch, retrieval-augmented prompting of a frontier model scored barely above zero [22]: the models could write plausible patches, but had no way to find the right file or check their own work. The first agentic scaffolds (SWE-agent, AutoCodeRover, Devin) moved the frontier by an order of magnitude within months, without any change to the underlying model. All they added were tools, a repository to navigate, and tests to run against. The 2024 scaffolding race that followed showed how much of that gain came from scaffolding rather than from reasoning capacity: a deliberately simple localise-then-repair pipeline such as Agentless stayed competitive with far more elaborate agentic loops.

A headline resolve rate hides where that progress actually landed. Figure 2 therefore splits Verified by the difficulty its human annotators assigned, tracking nine leaderboard systems from April 2024 to March 2025.

Three things stand out, and none of them is visible in the overall curve. First, the aggregate score is dominated by composition. Easy and medium tasks are 91% of Verified, so the headline number mostly reports how well an agent handles work a developer would finish within an hour. Second, the tiers improved at very different rates. Across the nine systems the easy tier rose from 27.3% to 80.4%, a factor of 2.9, while the medium tier rose from 10.0% to 62.1%, a factor of 6.2. The easy tier is now near its ceiling, so almost all

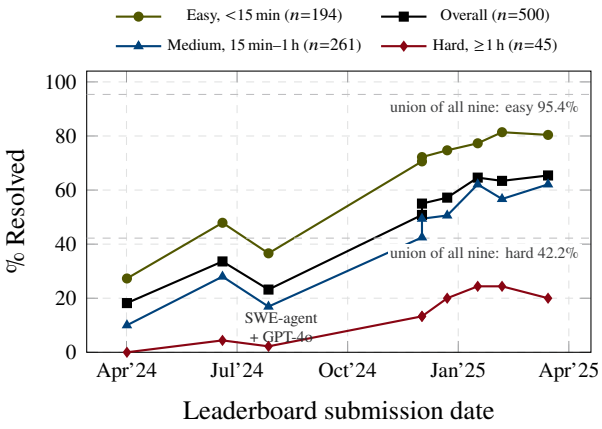

Figure 2 SWE-bench Verified resolve rate over time, overall and split by annotator-estimated difficulty, for the nine systems whose per-tier results are published. Rates and tiers come from one re-analysis of the leaderboard's predictions [15] against the human annotations shipped with Verified [31], so the series are directly comparable; points sit at each submission's date in the result archive [38]. The curves connect submissions in date order and are *not* a running frontier, which is why they dip in July 2024, when SWE-agent was resubmitted with a weaker backbone. Dashed lines mark the *union* over all nine: instances solved by at least one. The overall series is the tier-weighted average of the other three, except for SWE-agent + Claude 3 Opus, whose headline 18.2% (91/500) exceeds the sum of its own per-tier counts (79/500).

recent movement in the overall score is coming from the medium tier. Third, the hard tier behaves differently again: it climbed from 0% to about 20% by the end of 2024 and then stopped, with the last three systems scoring 24.4%, 24.4% and 20.0%. A system can gain ten points overall while solving no additional hard task, which is what makes the overall number a poor proxy for progress on the tasks that matter most.

The union sharpens this. Pooled across all nine systems, some agent solves 95.4% of easy and 84.3% of medium instances, but only 42.2% of hard ones, and just 9 of the 25 hard multi-file issues [15]. The remaining easy and medium headroom is therefore largely an ensembling problem: different scaffolds already solve different instances, which is why the inference-time scaling of §6 pays off. The hard tier is not an ensembling problem, since more than half of it defeats every system at once. Whether that reflects a capability ceiling or a retrieval blind spot common to all nine is a question the union cannot settle, and §6.4 takes it up.

Two caveats bound the figure. First, it plots nine submissions in date order, not a running frontier, and it

stops in March 2025 because the re-analysis behind it does. The July 2024 dip shows why this matters: the same SWE-agent scaffold resolves 33.6% with a Claude 3.5 Sonnet backbone in June and 23.2% with GPT-4o six weeks later, so scaffolding is not the only thing that moves these curves. Second, contamination sits on top of everything. Half the issues predate 2020 and come from repositories that are staple training data [13], while Lite and Full now receive few submissions at all [27]. §6.2 quantifies the resulting gap against Pro.

Pro and Multilingual (Table 2) answer the contamination and language critiques from within the SWE-bench format; a separate line of work leaves the format altogether and scores agents by the compute they spend [14]. §6.2 treats the landscape in full.

4 Planning and Search Strategies

Planning is the stage in which an agent turns a vague intent into an operational hypothesis about *what* code should change and *how*. In practice it is rarely a single up-front step: agents plan while reading issue reports, searching repositories, choosing files, sampling patches, and revising after failed tests. Planning is therefore best understood as a control layer over generation rather than a document written before generation begins. This section treats intent capture (§4.1), plan representations, search, and the plan–generate interface. Unlike §3, the mechanisms below are studied across function-level benchmarks (HumanEval, MBPP), competitive programming (CodeContests), and repository repair alike.

4.1 Intent Capture: The Open-Loop Stage

Intent enters as a GitHub issue, a bug report, or a sentence typed into a chat box. It is the only stage of Figure 1 that no surveyed system instruments, and the asymmetry is not obviously the right design.

Intent quality dominates outcomes. The clearest evidence comes from the benchmarks. SWE-bench Verified was built by having human engineers discard instances whose issue text was *underspecified*, that is, where the intent did not determine the patch [31]. Screening the intent, while changing neither the repositories, the agents, nor the grading harness, raises resolve rates by tens of points (§3.3). The same lever appears with the opposite sign in Lite, where some issue descriptions leak the fix outright, so that the intent contains its own solution [46]. An agent's score is, to a degree rarely acknowledged, a score on the issue-writing of the repository it was sampled from.

No end-to-end agent closes the loop here. Every other stage regulates on an external signal: planning against repository structure, generation against a parser and a linter, execution against a test runner, refinement against the resulting failures. Intent capture, as the surveyed systems implement it, has none. None of the five systems of §3.2 asks a clarifying question when the issue text is ambiguous; each commits to a reading and proceeds. OpenHands and Devin admit human interjection, but the initiative is the human's. In control terms the stage is *open-loop*: the model's reading of the issue simply *is* the specification, unchallenged.

The mechanism is not unknown. TiCoder [23] pins down intent through generated tests that the user accepts or rejects. ClarifyGPT [28] detects ambiguity by checking whether the model's own completions disagree, and asks only when they do. Vijayvargiya et al. [39] carry the idea to repository scale. They build an underspecified variant of SWE-bench Verified, and an agent that queries a simulated user does substantially better on it than one that guesses. They also find that frontier models left alone cannot tell specified from underspecified requests, and default to assuming rather than asking. The gap is therefore one of *transfer, not invention*. A clarifying question put to a human is an external signal in exactly the sense of §5.1: the human can contradict the model, and no other stage's oracle can.

The benchmark explains why the omission persists. SWE-bench is built from issues that a real merged PR resolved, so by construction the intent was good enough, and Verified deliberately removed the instances where it was not. An agent is therefore never rewarded for asking, and never penalised for guessing wrong about a genuinely ambiguous request. The clarification literature had to build a *new* benchmark [39] before the stage could be measured at all, which is the strongest evidence that the omission is an artefact of evaluation rather than of difficulty.

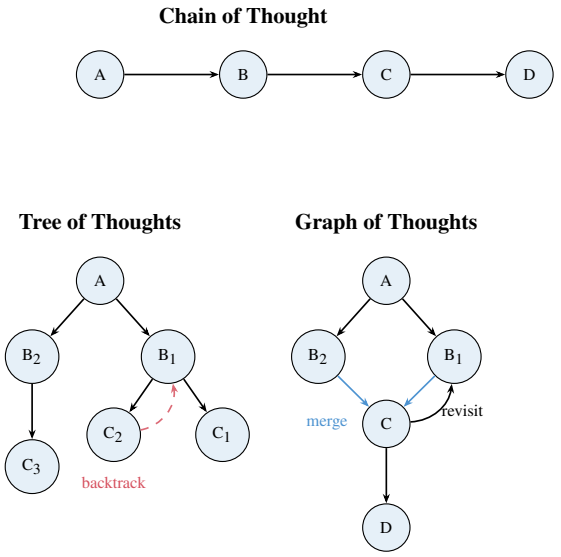

Figure 3 Plan representation structures. Chain of Thought commits to a single linear path. Tree of Thoughts explores alternatives and backtracks. Graph of Thoughts merges branches and revisits earlier states.

4.2 Plan Representations and Decomposition

The simplest plan representation is chain of thought [44]: a linear sequence that commits early to one trajectory. If the first design choice is wrong, every later step elaborates the mistake. Tree of Thoughts [49] and Graph of Thoughts [5] relax this by allowing exploration, backtracking, and merging (Figure 3). The mechanism in Tree of Thoughts is worth naming precisely, because the figure's "backtrack" arrow understates it. The tree is expanded under a classical search policy, and an LLM *value function* scores each partial state, so branches are pruned by lookahead rather than abandoned once they fail. The evaluator, not the branching, is what makes the structure useful. It is also the part a repository-scale agent finds hardest to supply: what is a half-finished patch worth, before any test has run? The difficulty is cost, not impossibility. SWE-Search [3] builds such an evaluator, a hybrid LLM value function that scores partial trajectories inside a Monte Carlo tree search, and gains 23% relative. But it must invoke a language model at every node, which is why it has not displaced the cheaper pipelines of §3.2.

Repository-level agents add a more concrete representation: *localisation plans*. Agentless breaks issue resolution into files, classes or functions, and edit locations before generating a patch [46]. AutoCodeRover plans over program structure through AST-backed search APIs [51]. Both name concrete program elements that can be checked against the repository, which turns a broad intent into a bounded search problem. Plans may also take the form of *specification sketches*: partial interfaces, pseudocode, assertions, or examples. What the useful ones share is that they are grounded in observable artefacts rather than in invented requirements.

4.3 Search and Exploration Strategies

Once a representation exists, an agent must decide how broadly to search. Single-sample generation treats the model's first completion as the solution; search-based systems spend extra inference to explore alternatives. The simplest form is self-consistency: sample several reasoning paths and take the most common answer [42]. Moving this to code forces the idea to change, and the reason is instructive. Self-consistency votes over reasoning paths that end in a *single canonical answer*, so the votes can be counted by string equality. Programs have no canonical form. Two correct patches may share no token, and two near-identical patches may differ in meaning. The vote must therefore be taken in a different space. Systems recover it in one of two ways: *behaviourally*, by clustering candidates on their outputs over a set of inputs (AlphaCode [24], CodeT [7]), or *syntactically up to normalisation*, by comparing candidates after AST canonicalisation (Agentless [46]). Sample-and-rank is thus self-consistency with an equivalence relation borrowed from program semantics rather than from string matching, and the quality of that relation bounds what the vote can buy.

AlphaCode generated up to millions of candidates for competitive programming, filtered and clustered them by execution results, then submitted a small diverse set. CodeT generates both programs and tests, and keeps the code that agrees with the generated tests. The key insight is that the right solution is often somewhere in the model's sample distribution even when the top-ranked output is wrong. The limitation is that the objective can be self-referential: if the generated tests are weak, selection optimises for the model's own misunderstanding. Execution-guided search is strongest when the ranking signal is an external oracle, and it decays toward noise as that signal is replaced by the model's own artefacts.

Search also appears in repository navigation. There it ranges not over complete programs but over *where*

to look and *what evidence to include*: a patch is unlikely to be correct if the agent never puts the relevant function, invariant, or failing trace in context. The trade-off is cost, since more samples multiply latency, tokens, and validation. Search is most useful when the ranking signal is at least as reliable as the generator is diverse.

4.4 Plan–Generate Interplay

Planning and generation can be coupled in three ways. *Plan-then-generate* plans once and conditions generation on the result: easy to inspect, brittle if the plan is wrong [46]. *Interleaved planning* alternates plan updates and tool use inside the observe–think–act loop [50]: adaptive, but the plan is distributed across many turns and is hard to audit. *Generate-then-refine* treats the first output as a hypothesis and revises it via critique or execution feedback [26], which connects planning to §5.

The central design question is not whether agents should plan, but how plans stay *grounded*. Grounding comes from retrieval over repository structure, AST-level symbol search, execution traces, failing tests, and explicit constraints pulled out of the issue. Effective systems keep plans close to these artefacts and treat generation as one move in a larger search. That reframes code synthesis from "write the answer" to "maintain and test a hypothesis about the required program change".

This distinction resolves what would otherwise be an awkward result. Agentless, the system that plans *least*, is competitive with those that plan most (§3.2). If the claim were that deliberation helps, Agentless would refute it. The claim is instead that *grounding* helps, and deliberation is only one way to get it, often an expensive one. Agentless gets its grounding structurally, by hard-coding the hierarchical narrowing that an agentic loop has to rediscover by exploring. It therefore confirms the argument rather than undermining it. The corollary is that finding the right location is harder than editing it once found. AutoCodeRover's SBFL variant changes nothing about repair, only about localisation, and it buys three points of resolve rate plus seven instances no other configuration reaches [51]. Planning here is not reasoning about *how* to write code. It is reasoning about *where* the code is, and about what evidence would confirm that guess.

5 Execution Feedback and Iterative Refinement

Code generation rarely succeeds in one shot. Agentic systems execute generated code, observe runtime outcomes, and revise. While §3 scores agents on SWE-bench, refinement mechanisms are often evaluated on function-level suites (HumanEval, MBPP), text-to-SQL (Spider), and APR benchmarks (Defects4J).

One distinction organises the whole section: *what closes the loop*. A refinement mechanism is a control loop only if some signal outside the model's own judgement can contradict the model. The literature usually treats this as a binary, external feedback against self-critique. That binary has no room for the mechanism behind most of the field's recent gains, so we split it into three positions.

An *authoritative oracle* (a test runner, a compiler, a type checker, a human) is ground truth. It can surprise the model, and refinement against it reliably improves code. A *learned or generated proxy* (a trained critic, model-generated tests, an LLM judge) sits one step removed. It carries real information, because ground truth went into it at training or generation time, and that is why it works. But at inference it is a model, not an oracle, so it cannot surprise anything. Its ceiling is how faithful the proxy is, which is the same bound §4 derived for sample-and-rank. Finally, *same-distribution self-critique* draws the critic from the very distribution that produced the error. The loop closes onto itself, and the control is only apparent.

Table 3 sorts the mechanisms on this axis, and §5.3 gives the evidence that this is the axis that matters. The split earns its keep immediately. The two most-cited positive results below, Reflexion's HumanEval headline and OpenHands's inference-time scaling, both sit in the middle column, and reading them as oracle-closed overstates what they show.

5.1 Feedback Signals and Environments

Agents consume five broad signals. The useful way to sort them is not by form but by whether they can tell the model something it does not already believe. *Test results* and *compiler or interpreter errors* are oracles: an artefact outside the model computes them, and they are indifferent to the model's confidence. *Static analysis* is oracular too, and unusually cheap, because

Table 3 Refinement mechanisms sorted by what closes the loop. **O** = authoritative oracle (can contradict the model); **P** = learned or generated proxy (carries information, but is itself a model and cannot surprise); **S** = same-distribution self-critique (can only reorganise what the model already believes).

Mechanism	Closed by	Signal	Characteristic failure
Self-Refine [26]	S	Model critiques own output	Plateaus; no signal can contradict the generator
Self-debugging, no tests [8]	S	Self-generated explanation	Explains the bug as intended behaviour
Self-debugging, with tests [8]	O+S	Test verdict + model diagnosis	Bounded by test coverage
Reflexion [36]	O or P	Task reward where one exists; on HumanEval, self-generated tests	Usually filed as external, but its code result rests on a proxy
SelfEvolve [21]	O	Interpreter errors, traces	Silent (non-crashing) semantic bugs
D4C [47]	O	Failing tests + full function	Plausible-but-incorrect patches
CRITIC [17]	O	Self-critique routed *through* an external tool	Needs a tool covering the error class
Agentless validate [46]	O+P	Regression tests + generated reproduction tests	The proxy half inherits the model's misreading
Best-of-N + critic [33]	O+P	Executed rollouts ranked by a trained critic	Critic is a distilled proxy; cost scales with N

it reports before execution. *Human feedback* is the most authoritative signal and the least available. *Self-critique*, in which the model explains its own code and looks for discrepancies ("rubber duck debugging") [8], is the only one of the five that the system under test produces itself.

The three positions differ in cost as well as in authority. An oracle demands infrastructure and may be slow. Self-critique is fast and needs no setup, but an agent that could reliably tell its wrong code from its right code just by looking would have written the right code in the first place. The proxy's failure is the specific one: a true oracle is wrong only if the test suite is wrong, whereas a proxy is wrong exactly where the model was wrong to begin with, which is where it is needed most.

To run untrusted code safely, systems sandbox it: SWE-agent and AutoCodeRover execute each task in a per-instance Docker container, which is what makes SWE-bench's environment-dependent suites reproducible at all, and OpenHands adds a browser and an IPython server in the same sandbox. Sandboxing serves two purposes that are easily conflated: *reproducibility*, ensuring the harness scores the patch and not the environment, and *containment*, preventing generated code from damaging the host. Most research systems are engineered for the former; the latter matters once agents run against real infrastructure (§6.4).

5.2 Iterative Refinement Mechanisms

Reflexion. Reflexion [36] treats refinement as reinforcement learning without weight updates. After each trial the agent writes a natural-language critique of what went wrong, stores it in an episodic memory buffer, and retrieves it on later attempts. The innovation is turning feedback into *verbal* form that persists across trials. On HumanEval, Reflexion reports 91% pass@1 against 80% for single-shot GPT-4. What closes the loop in that number is routinely misread. Reflexion is usually described as learning from an *external* reward, and on tasks that supply one it does; but HumanEval supplies no such reward at inference, so Reflexion writes its own unit tests and reflects on their verdicts. The signal is a generated proxy, and it inherits exactly the failure Table 3 assigns to Agentless's generated tests.

Self-debugging. Chen et al. [8] have the model explain its own code line-by-line. The explanation acts as a self-generated specification, and discrepancies against it become visible. Where execution feedback is available, the model also reasons about error messages before proposing fixes. On Spider, which supplies no unit tests, self-debugging improves accuracy by 2–3%. On benchmarks that do supply tests (TransCoder, MBPP) the reported improvement reaches 12%. The gap between those two numbers is the point. A small but real gain survives with no oracle at all, so pure self-critique is not worthless; a gain several times larger appears as soon as an oracle is admitted.

5.3 Does Refinement Work? The Limits of Self-Correction

The results above are usually read as evidence that agents improve their own code. Three lines of work complicate that reading, and taking them seriously sharpens rather than weakens the control-layer thesis.

Self-repair is partly inference-time scaling in disguise. Olausson et al. [30] observe that a self-repair loop does not spend a fixed budget. Producing a critique and a revision costs tokens that could have drawn more independent candidates instead. Once the budget is held fixed, and the loop is compared against simply sampling k programs i.i.d. and taking the best, much of self-repair's apparent gain disappears. What survives comes from the *feedback*, not from the iteration: repair helps most when the model receives a stronger critique than it could produce for itself, and in their strongest condition that critique comes from a human. Reflexion's HumanEval headline should therefore be read as a result about a *budget* as much as about a mechanism, and compared against a best-of-N baseline at matched cost.

Intrinsic self-correction does not reliably help. Huang et al. [19] isolate the case where no external signal is available. They find that models asked to revise their own answers often make them worse, turning correct answers into incorrect ones somewhat more often than the reverse. Their evidence is on reasoning benchmarks (GSM8K, CommonSenseQA, HotpotQA) rather than on code, and we flag that rather than quietly generalising. What transfers is the mechanism, not the measurement. The critic and the generator are the same distribution, so the critique cannot add information the generator lacked, and nothing in that argument is specific to arithmetic. On code, the load is carried by Olausson et al.

The constructive converse. An argument that rests only on what breaks is weaker than one that also predicts what works. CRITIC [17] supplies the missing half. Holding the model fixed, self-critique that may consult an external tool corrects reliably, while self-critique that may not yields little or nothing. The variable that moves the outcome is neither the model nor the prompt. It is the presence of something outside the model in the loop, which is the axis of Table 3 isolated experimentally. CRITIC's own limit is instructive: a tool helps only for errors it can detect, the same coverage bound that weak test suites impose in §5.5.

What this implies. Refinement is a control layer when a signal the generator did not produce closes the loop. When no such signal does, it yields only small and unreliable gains. Every mechanism that survives contact with repository-scale evaluation (Agentless's validation phase, SWE-agent's edit–execute loop, D4C's test-driven repair) is closed externally. The purely internal ones cluster in function-level benchmarks, where a comprehensive suite is available to the *evaluator* even though the agent cannot see it.

This also unifies two techniques the literature treats separately. Iterative self-repair and best-of-N with a reranker are the same lever: a budget of extra inference spent to reduce variance, pulled either sequentially or in parallel. Once the budget is fixed, the question is not "does the agent refine?" but "which allocation of the same tokens buys more correctness?" OpenHands answers in one direction. Five parallel rollouts reranked by a dedicated critic reach 66.4% on Verified, where a single rollout reaches 60.6% [33]. The critic is a *separate, trained* model rather than the generator judging itself, and that separation is what makes the gain possible. But the critic is a proxy, not an oracle, so this is not simply "external feedback wins". Olausson et al.'s strongest condition was a human critic, and the gain here is bought by ranking rollouts that were themselves executed, that is, by an oracle and a proxy together.

5.4 Fault Localisation and Self-Repair

When refinement targets bugs in existing code rather than draft solutions, it becomes *automated program repair* (APR), classically staged as fault localisation, patch generation, and validation. Agents challenge this structure. RepairAgent [6] shows that the observe–think–act loop transfers wholesale: driven by a finite-state machine over a small tool set, it localises, edits, and tests without a separate localisation phase, repairing 164 Defects4J bugs of which 39 had resisted every prior technique.

Pre-LLM APR relies on *spectrum-based fault localisation* (SBFL), ranking lines by suspiciousness derived from test coverage (Ochiai, Tarantula, DStar). But even oracle-provided bug locations do not guarantee correct repairs. D4C [47] argues that LLMs should repair entire functions rather than isolated hunks, aligning the task with their infilling pre-training objective: on Defects4J it repairs 180 single-function bugs with 10 samples each, outperforming systems with perfect fault localisation by 10%.

Reconciling FL-free repair with localisation-first agents. Read beside §4, D4C appears to contradict the systems surveyed there. Agentless and AutoCodeRover treat localisation as the core of the problem, while D4C reports that it is unnecessary and even

harmful. Both are right, because they work at different scales, and naming the scale is more useful than picking a winner. D4C's Defects4J tasks are *single-function* bugs: the buggy function is handed to it. So what D4C removes is not the search for the faulty function. It removes the finer-grained ranking of *lines within* a function already known to be faulty, and its argument, that line-level targeting fights the model's infilling prior, is convincing at that granularity. A SWE-bench agent faces an earlier question: which function, among tens of thousands? No pre-training prior answers it, and the relevant symbol may not appear in the issue text at all. The reconciliation is therefore a claim about scale. *Below* the function boundary, localisation is a constraint the model does not need and should not be given. *Above* it, localisation is the task. This is why AutoCodeRover's SBFL variant helps by ranking functions, while D4C's ablation of perfect fault localisation helps by un-ranking lines. The two results look opposed, but they measure opposite sides of the same boundary.

One special case decides whether localisation is a problem at all. When the fault crashes, the stack trace localises it for free, and systems such as Self-Evolve [21] simply feed the trace back to the model. When it does not crash, as with an off-by-one that silently returns the wrong answer, there is no trace, the oracle reports only a failed assertion, and the agent is thrown back on search. That is why an interpreter is a weaker oracle than a test suite.

5.5 Convergence and Stopping Criteria

Iterative refinement must terminate, balancing quality, cost, and the risk of overfitting to weak test suites.

The simplest criterion is *full test pass*. This works for competitive programming, where suites are comprehensive. In repository tasks the tests may be incomplete: a patch that passes them all may still be wrong, and a correct patch may fail for environment reasons. The criterion is also, quietly, an oracle. A loop that halts exactly when the answer becomes right has smuggled ground truth into its stopping rule, and any self-critique running inside such a loop will look more effective than it is.

A second criterion is *diminishing improvement*: if iterations 3, 4, and 5 all fail the same test with the same error, more iterations are unlikely to help. The empirical basis is the shape of the repair curve. Olausson et al. find that the marginal return of another repair round falls off quickly, and that once the token budget is held fixed it is often negative compared with drawing a fresh independent sample [30]. Where the loop is closed only internally, Huang et al. find it can be negative outright [19]. A stopping rule and a budget allocation are therefore the same decision, and "iterate until it passes" is defensible only when an external oracle defines *passes*. Sequential refinement and parallel best-of-N draw on the same pool of tokens, so fixing the budget is what makes the two comparable at all, and an iteration cap is really a bet about which of them turns tokens into correctness more efficiently.

Overfitting and the human oracle. If the suite is weak, the agent may write code that narrowly passes it and fails on unseen inputs: the "plausible patch" problem. The mitigations sort onto the axis of §5.1, and the sorting is more useful than the list. Strengthening the oracle genuinely helps: static checks add a second, independent artefact that the patch must also satisfy, and they are cheap. Strengthening the *proxy* helps less: generating more tests inherits whatever the generator misunderstood, which is exactly where the extra tests are needed. Asking the model to predict whether its own patch generalises is not a mitigation at all; it is the open loop of §5.3 under a new name. No technique eliminates overfitting, and the reason is structural. The oracle is a proxy for correctness, and optimising against a proxy is what overfitting *is*. Where no automated oracle is good enough, the remaining move is to put a human in the oracle's place, as OpenHands and Devin both allow; §6.4 argues that this restates the problem more than it solves it.

6 Discussion

We now consolidate what the three fronts share and which constraints bound all of them. Progress has come less from any single architectural idea than from *coupling* generation to external evidence, and that same coupling exposes the field's hardest problems: evaluation integrity, finite context, and cost.

6.1 Consolidated Patterns and Trends

Three recurring patterns, and what unites them. The *observe–think–act loop* [50] is the near-universal control structure (§3.1). Agentless's fixed pipeline [46] is the exception, and it stays competitive

by hard-coding the sequence the loop would otherwise have to discover. *Execution-grounded selection* ranks candidates by tests or regression preservation rather than by model confidence (AlphaCode [24], CodeT [7], Agentless), and it is the most reliable quality lever. And *planning acts on generation* rather than preceding it (§4). The second pattern explains the other two. Everything that works, works by putting something outside the model in a position to contradict it: a test runner, a compiler, an AST index, a human. Planning earns its place by grounding hypotheses in repository structure that can be checked. Refinement earns its place when a signal the generator did not produce closes the loop, and yields only small and unreliable gains when none does, gains that on reasoning tasks turn outright negative (§5.3). Read this way, the pipeline is not five stages of reasoning. It is a scaffold for arranging confrontations between a generative model and an environment that can say no. Its two weakest points are exactly the two places where nothing can: intent capture, whose available oracle is a human nobody queries (§4.1), and test-based validation, whose oracle is weak enough to be gamed (§6.2).

Agents regulate tightly on a proxy setpoint. Calling both stages a "control layer" compresses two different things, and the difference is worth drawing out. Refinement is a control loop in the literal sense: a signal is measured against a setpoint, and the output is revised. Planning is not. Agentless's hierarchical narrowing regulates against nothing. It is a feed-forward pass, and its contribution is to make the hypothesis *checkable* rather than to check it. What unites the two stages is not feedback but the placement of an external artefact between the model and its own confidence. And the setpoint that refinement does regulate against, "the tests pass", is a leaky proxy: patches that pass can still be overfitted, or the product of reward hacking (§5.5). Current agents are therefore control loops regulating tightly on a signal that only approximates what we want. Their impressive convergence and their characteristic failures are the same phenomenon seen from either end.

Macro trends, and their expiry date. The field has moved from single-shot prompting to agentic scaffolds, from flat chain-of-thought to structured search and localisation, and most recently to *inference-time scaling*, where trajectories are sampled and reranked by a critic (OpenHands's 60.6% → 66.4% [33]). The consistent finding across §3 is that *scaffold and interface matter as much as the backbone model*. That finding is time-stamped rather than timeless. A newer line of work moves the scaffold's competence into the weights, by reinforcement learning over software-evolution data [45] or by training agent and verifier together in an executable environment [34]. If this succeeds, the scaffolding advantage was a transitional artefact of using models that were never trained to be agents. Deployment has shifted in parallel. The systems surveyed here are research artefacts, whereas the agents developers actually use (Claude Code, Cursor, Codex) are products whose scaffolds are undocumented and whose evaluations are self-reported. That is one reason the integrity questions below have become urgent.

6.2 The Evaluation Landscape

A layered benchmark stack, straining at every layer. *Function-level* suites (HumanEval, MBPP) are effectively saturated (Reflexion reported 91% pass@1 on HumanEval already in 2023 [36]) and no longer discriminate among frontier systems. Part of that saturation is contamination rather than capability: LiveCodeBench [20], which admits only problems published after a model's training cutoff, finds performance drops on genuinely unseen problems, the same story Figure 4 tells one level up. *Repository-level* evaluation shifted the centre of gravity to SWE-bench [22], whose Verified split [31] is the de-facto standard, yet §3.3 already noted its difficulty skew (39% of tasks are trivial <15-min fixes and 86% touch a single file [15]), its repository monoculture (Django dominates), and its contamination risk [13].

Three responses have emerged. The first is *harder, contamination-resistant successors*. SWE-bench Pro [11] contributes 1,865 long-horizon problems from 41 repositories, and resists memorisation with copyleft and held-out proprietary code; at release, top systems scored ~23% against 70%+ on Verified. SWE-bench Multilingual [37] extends the format to 300 tasks across 9 languages. Figure 4 makes the divergence concrete: models at or above 80% on Verified fall to the mid-40s or high-50s on Pro, with Claude Opus 4.5 scoring 80.9% against 45.9% under Scale's standardised harness [11, 1]. Two confounds stop us reading the *size* of each drop as a measurement of memorisation, and we flag them because they are the confounds this report holds others to. Pro is harder as well as fresher, and its column mixes harnesses: only

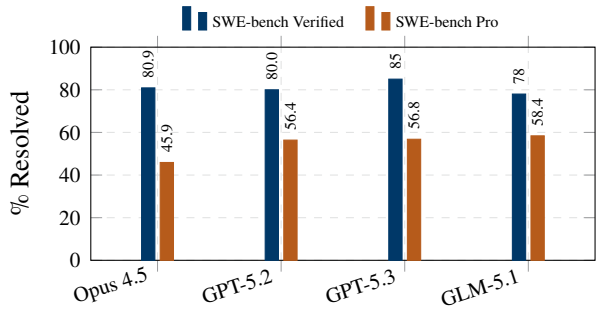

Figure 4 The Verified–Pro gap. The *same* model weights score at least twenty points lower on the harder, contamination-resistant SWE-bench Pro than on the saturated, widely-scraped Verified. Only that *floor* should be read here. Neither the per-model differences nor the width of the band should be: on Pro only Opus 4.5 is scored under Scale's standardised SEAL harness, the rest self-report with custom scaffolding, so the model under the strictest harness shows the largest drop for reasons unrelated to memorisation. Snapshot of April 2026; the Pro frontier has since moved. Sources: [11, 1].

Opus 4.5 is scored under Scale's standardised evaluation. Consistency then requires us to decline the quantity the figure invites us to quote. The width of the band *is* the per-model spread we have just said cannot be read, and its upper end belongs to the one model measured most strictly. What survives is a floor and a direction. Every model drops by at least twenty points, and no mechanism that would explain a drop that size is flattering. The signal was strong enough that OpenAI stopped reporting Verified in early 2026, citing near-universal contamination (models reproduce gold patches from the task ID alone) and flawed residual tests [32].

The second response targets *benchmark integrity directly*. A stricter harness that closes test-leakage channels cuts reported scores substantially, by double digits on Pro for some models. This prompted the observation that "reward hacking is swamping model intelligence gains": agents partly learn to exploit the grading oracle rather than to solve the task [10]. The third response is *efficiency-aware evaluation*. SWE-Effi [14] re-scores agents under resource constraints and finds that resolve rate decouples from cost. Agentless leads its study on resolve rate (48% with Qwen3-32B) but spends 83.1 API calls and 209.4 seconds per issue, so a single pass@1 number hides an order-of-magnitude spread in cost.

One number no longer suffices. Credible comparison now requires difficulty stratification, contamination controls, and cost normalisation reported alongside accuracy. The pipeline framing makes the coverage gap concrete: most benchmarks score only the *end* of the pipeline (did the final patch pass?), leaving plan quality, localisation precision, and refinement efficiency unmeasured.

6.3 The Context Window Bottleneck

Longer windows do not dissolve the bottleneck. Every stage is constrained by the same resource: finite working context. Real repositories exceed even million-token windows, so an agent never sees the whole program and must decide *what* to read. Longer windows do not settle that question. Repository-level studies find that keeping the full *dependency* context works best, while padding the context with loosely related code can mislead the model and hurt correctness [29]. More tokens are not uniformly better. A second failure mode is dynamic. Agents that append every observation to an ever-growing trajectory suffer *context explosion* and *semantic drift*, as critical early information gets buried [25]. Cost compounds with it. The whole prefix is re-sent each turn, so tokens processed over n turns grow as $O(n^2)$, which is why turn budgets are an efficiency lever and not merely a safety one [16]. That exponent is a property of the protocol rather than an empirical finding, and prompt caching (used by every deployed agent) cuts the constant by roughly an order of magnitude without changing the growth rate.

Context as a managed resource. The surveyed systems therefore treat context as a managed resource rather than a passive log: hierarchical repository skeletons with embedding retrieval (Agentless), AST search returning a class skeleton before any full method body (AutoCodeRover), and observation compression of older turns (SWE-agent). The current direction is to make context management *active*, summarising, pruning, and re-retrieving as a deliberate agent action [25]. Context quality, not model capability alone, frequently determines whether the correct patch is even reachable: an agent cannot fix what it never places in context.

6.4 Open Challenges

Three challenges cut across all three fronts and remain open.

Long-horizon, multi-file reasoning. Agents excel at small, local, single-file fixes but degrade steeply on tasks requiring coordinated changes across many files. On Verified the best single system resolves about 80% of the easy tier but only 24% of the hard [15], and the drop on the explicitly long-horizon SWE-bench Pro [11] points at the same ceiling.

Is this the same constraint as the context bottleneck above, or a different one? §3.3 showed that the easy and medium tiers behave like an *ensembling* problem, with the union over nine systems running far ahead of any one of them, while the hard tier does not. That contrast rules out a *sampling* limitation. It does not rule out retrieval. The nine systems share the issue text and a narrow family of localisation strategies, and three of them are the same SWE-agent scaffold on different backbones, so a blind spot common to all of them would look exactly like a reasoning ceiling.

We nonetheless expect training-time approaches [45, 34] to move the hard tier, rather than better retrieval. Hard instances are disproportionately multi-file (only 9 of 25 such issues are solved by anyone), and composing a coherent cross-file change is not something more context supplies. To make that a prediction rather than an expectation, it needs a discriminating test, and one is available: hold the retrieval pipeline fixed and hand the agent the gold file set, so that localisation is oracular by construction. If hard-tier rates then rise substantially, the ceiling was retrieval and we are wrong. If they do not, only training will move it. We are not aware of anyone having run this ablation, and it is the cheapest experiment we can name that would settle a question the field currently answers by assertion, ours included.

Specification ambiguity. This is the open-loop intent stage of §4.1, seen from the outside. Agents start from natural-language intent that is often underspecified. When the issue text omits edge cases or implicit invariants, the agent has to guess. And execution feedback only constrains behaviour that the available tests exercise, so a wrong guess about an untested invariant is never contradicted and is scored as a success. Closing the gap between an ambiguous intent and a verified specification is not an unstudied problem (§4.1): clarification, test synthesis, and interactive intent capture all exist and all work [23, 28, 39]. What is missing is adoption. No end-to-end system surveyed here queries a user, and the incentive to add one is absent by construction: a dataset built from issues that a real pull request resolved is a dataset with the ambiguity already filtered out. The challenge is therefore not to invent the mechanism but to build an evaluation that would reward using it.

Cost and efficiency. The dominant quality lever, inference-time scaling with critic reranking, directly multiplies cost, while trajectory length inflates tokens per turn [16]. Because resolve rate decouples from compute [14], the practical frontier is not peak accuracy but accuracy per dollar and per minute.

6.5 Implications for Software Engineering Practice

These trends shift the developer's role rather than remove it. As agents absorb localisation, editing, and test-driven repair, the human contribution moves upstream (precise intent and acceptance criteria) and downstream (reviewing and accepting patches). First, *verification becomes the bottleneck*. Benchmark scores partly reflect reward hacking [10], and validation oracles are weak enough to admit plausible-but-wrong patches (§5.5), so a green test run is not acceptance. Stronger oracles (regression suites, human review) become first-class pipeline stages. The cost of that review is easy to underestimate, and it has now been measured. In a randomised controlled trial, experienced open-source developers took 19% *longer* on their own repositories when allowed to use early-2025 AI tools, while estimating afterwards that the tools had made them 20% faster [4]. One study on a narrow population settles nothing, but the direction of the error is instructive: the speed-up in generation is visible and the added verification burden is not, so even the people best placed to notice the trade did not notice it. Second, *execution infrastructure becomes central*. Sandboxes, reproducible environments, and CI integration are what make execution-grounded selection possible at all. The economic frame shifts with them, from "can an agent resolve this issue?" to "at what cost, and with what assurance?"

7 Conclusion

This report surveyed autonomous AI agents for code generation through a single lens, the five-stage agent pipeline, and synthesised three research fronts that instantiate its stages. End-to-end systems (§3) converge on an observe–think–act loop over a purpose-built agent–computer interface, with repository-scale

context managed by retrieval, AST search, or compression; that Agentless rivals interactive agents at lower cost shows autonomy is a design choice, not a prerequisite. Planning (§4) contributes not deliberation but *grounding*, which is why the least deliberative system remains competitive. Execution feedback (§5) shows that running the code, not generating it, supplies most of the reliability.

The pipeline framing makes the central coupling visible, and sharpens it into something testable. Planning and refinement keep generation tethered to external evidence, but only where a signal the generator did not produce can contradict it. Between the oracles that can contradict the model and the self-critique that cannot lies the proxy: a trained critic or a generated test, which carries real information yet is bounded by its own fidelity (§5.3). Much of what the field reports as successful self-improvement lives in that middle band. Read this way, the pipeline's two weaknesses are the two stages with no oracle behind them: intent capture, whose only oracle is a human nobody queries (§4.1), and validation, whose oracle is weak enough to be gamed (§6.2). The question at the frontier has changed accordingly, from "can an agent resolve real issues?" to "at what cost, and how far can the result be trusted?"

Each promising direction amounts to supplying an oracle where one is missing. Contamination-resistant benchmarks, reported with cost and difficulty stratification, and benchmarks that score the pipeline's intermediate stages rather than only its final patch. A decision, by the ablation of §6.4, on whether long-horizon reasoning is a training problem or a retrieval one. And the adoption at repository scale of the interactive intent capture that already works at function level. Agentic code generation has clearly crossed from autocomplete to autonomous issue resolution. Whether it crosses from impressive to dependable will be decided by whether we learn to let the environment say no to the model as reliably as a reader once did, because on the evidence of this report the model will not say it to itself.

References

[1] AgentMarketCap. SWE-bench pro vs verified: The gap that rewrote 2026 agent procurement. https://agentmarketcap.ai/blog/2026/04/16/swe-bench-pro-vs-verified-25-point-gap-procurement-framework, 2026. Paired Verified/Pro leaderboard data, compiled April 2026. Accessed: 2026-06-30.

[2] Answer.AI. Thoughts on a month with Devin. https://www.answer.ai/posts/2025-01-08-devin.html, 2025. Accessed: 2026-07-09.

[3] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024. ICLR 2025.

[4] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. METR randomised controlled trial.

[5] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

[6] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. arXiv:2403.17134.

[7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

[8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.

[9] Cognition AI. Introducing devin, the first AI software engineer. https://cognition.ai/blog/introducing-devin, 2024. Accessed: 2026-07-09.

[10] Cursor. Reward hacking is swamping model intelligence gains. https://cursor.com/blog/reward-hacking-coding-benchmarks, 2026. Accessed: 2026-06-30.

[11] Xiang Deng et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[12] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. A survey on code generation with LLM-based agents. *arXiv preprint arXiv:2508.00083*, 2025.

[13] Epoch AI. What skills does SWE-bench verified evaluate? https://epoch.ai/publications/what-skills-does-swe-bench-verified-evaluate, 2025. Accessed: 2025-06-01.

[14] Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M. Zhang, Pinjia He, and Ahmed E. Hassan. SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.

[15] Jatin Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html, 2025. Blog post, accessed: 2025-06-01.

[16] Pengfei Gao and Chao Peng. More with less: An empirical study of turn-control strategies for efficient coding agents. *arXiv preprint arXiv:2510.16786*, 2025.

[17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.11738.

[18] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024.

[19] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024.

[20] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2403.07974.

[21] Shuyang Jiang, Yuhao Wang, and Yu Wang. SelfEvolve: A code evolution framework via large language models. *arXiv preprint arXiv:2306.02907*, 2023.

[22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

[23] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022. TiCoder.

[24] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[25] Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.

[26] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[27] Matias Martinez and Xavier Franch. Dissecting the SWE-bench leaderboards: Profiling submitters and architectures of LLM- and agent-based repair systems. *arXiv preprint arXiv:2506.17208*, 2025.

[28] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1:2332–2354, 2024. arXiv:2310.10996.

[29] Nam Le Hai Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[30] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *The Twelfth International Conference on Learning Representations*, 2024.

[31] OpenAI. Introducing SWE-bench verified. https://openai.com/index/introducing-swe-bench-verified/, 2024. Accessed: 2025-06-01.

[32] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/, 2026. Accessed: 2026-06-30.

[33] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model, 2025. Accessed: 2026-05-28.

[34] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*, 2024.

[35] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, 2024.

[36] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[37] SWE-bench Team. SWE-bench multilingual. http://www.swebench.com/multilingual.html, 2025. 300 tasks, 9 languages, 42 repositories. Accessed: 2026-06-30.

[38] SWE-bench Team. SWE-bench leaderboards. https://www.swebench.com. Underlying evaluation artefacts at https://github.com/SWE-bench/experiments, 2026. Accessed: 2026-07-09.

[39] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. Interactive agents to overcome ambiguity in software engineering. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. arXiv:2502.13069; later revised as "Ambig-SWE".

[40] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

[41] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Graham Neubig, et al. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741.

[42] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[43] Yanlin Wang et al. Agents in software engineering: Survey, landscape, and vision. *arXiv preprint arXiv:2409.09030*, 2024.

[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[45] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and

Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[46] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[47] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. Aligning the objective of LLM-based program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. D4C. arXiv:2404.08877.

[48] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[49] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.

[51] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.